

VoltSentry — Presenter Guide

How and when to explain what. A 4-minute run-of-show, a slide-by-slide script, the live-demo runbook, recovery lines, and Q&A prep.

The one sentence: *"VoltSentry is a transparent software proxy that sits on the wire between an EV charger and its management backend, inspects every OCPP message in real time, and quarantines a compromised charger before it can commit billing fraud or destabilise the grid — including the slow data-drift attack that no rule engine can catch."*

Roles & setup

ROLE	OWNS
Narrator	Talks the whole time. Never touches the keyboard. Owns the story and the Q&A.
Driver	Advances slides, switches to the dashboard, points the cursor at what the narrator is describing.
Operator	Presses the attack keys on cue (1 2 3 4 , r between each). Watches the terminals, calls out if a process dies.

Pre-flight checklist (do before you walk up)

- Start order, one terminal each: `python -m simulator.csms` → `python -m proxy.main` → `python -m simulator.twin` → `cd dashboard && npm run dev` → `python -m cli.attack`. The proxy dials upstream at handshake, so CSMS must be up first.
- Dashboard on screen, logged in (Demo Quick-Login), fleet charging, 8 stations green, transformer gauge steady.
- Click **Start Monitoring (enable audio)** once — browser autoplay policy blocks the alert sound until a click.
- Browser at 100% zoom, dark mode, full screen, OS notifications off.
- **Backup video** of one clean 4-attack run open in a background tab. If anything wedges mid-demo, cut to it and keep narrating.
- Terminals tiled where judges can see the `[proxy]` / `[twin]` prefixes — they are part of the proof.
- Run the full sequence once, cold, five minutes before you present.

The 4-minute timeline

TIME	ON SCREEN	BEAT
0:00	Deck slide 1-4	Problem → threat model → the approach. Fast.
1:00	Deck slide 5-7	Architecture, then the two detection tiers. Land "the residual feature."
1:45	Dashboard	Switch to the live NOC. Fleet charging.
2:00	Dashboard	Press 1 — meter_spoof. CP-03 quarantined.
2:30	Dashboard	r then 2 — oscillation. Transformer gauge swings.
2:55	Dashboard	r then 3 — session shadow. Handshake refused.
3:15	Dashboard	r then 4 — subtle drift. No rule fires. Risk Index climbs past 0.65.
3:45	Dashboard	Click Export incident logs — JSON downloads.
3:55	Deck slide 9 / 12	"What's real" honesty beat, then the close.

Slide-by-slide script

Talking points are prompts, not a script to read aloud. Say them in your own words.

1 Title

One point: what this is, in one breath.

- "EV charging networks run on a protocol called OCPP — JSON messages over a WebSocket between the charger and a central server."
- "VoltSentry is an inline proxy that reads every one of those messages and stops a compromised charger before the damage reaches the backend or the grid."

2 The problem

One point: this is exposed infrastructure, and today's defences are too slow or too expensive.

- Chargers are roadside hardware on weakly-authenticated connections — soft edge nodes on the energy grid.
- Cloud platforms analyse this data in batches, minutes to hours later — after an attack has already landed.
- You can't retrofit a firewall chip to millions of installed chargers. So the fix has to be pure software, and it has to be inline.

3 Threat model

One point: four concrete attacks, and the fourth is special.

- Meter spoofing — bill near zero while real power flows. Grid oscillation — synchronised start/stop to trip a transformer. Session shadowing — hijack a live charger's session with a duplicate handshake.
- "And the fourth — subtle drift — is a slow, compounding lie that never breaks a single threshold. Hold that one in mind; it's why the second detection tier exists."

4 The approach

One point: transparent, inline, drop-in — the CSMS never knows it's there.

- A reverse proxy on the wire. Clean traffic is relayed straight through, both directions, untouched.
- Every frame is parsed, the session state machine is enforced, and the detection engine scores it before it's forwarded.
- On a hard violation: drop the frame, tell the charger to go unavailable, close the socket. Sub-millisecond design budget, zero hardware change.

5 Architecture

One point: one process on the wire, feeding one live dashboard.

▢▢ – Trace the diagram left to right with the cursor: fleet → proxy → CSMS, and the side branch proxy → dashboard + forensic log.

- Station identity comes from the URL path, parsed at handshake — the proxy opens a matching upstream socket per charger.
- OCPP replies don't carry the action name, so the proxy keeps its own call-correlation map. That's what makes the state machine possible.
- The dashboard feed is a separate WebSocket on its own port — the browser talks to it natively.

6 Tier-1 — deterministic rules

One point: six rules, zero latency, they catch everything loud.

- "These are physics and protocol invariants — power can't exceed the charger's rating, a session can't send meter values before it's authorised, the same charger ID can't hold two live sessions."
- Point at [R6_METER_FRAUD](#): "This one watches the reported energy total pull away from the reported power. Honest sessions keep those in lock-step; a skimming charger can't."
- "Three of these quarantine the station outright — R2, R5, R6."

7 Tier-2 — the drift problem

One point: the residual feature is the whole trick.

- "A slow drift stays inside every threshold — so it's not an outlier, and a rule engine is blind to it by construction."
- "We give the model one engineered number: the reported energy total, minus the integral of reported power over time. Honest sessions hold that at zero. Any lie makes it climb — and it only ever climbs."
- "Two models score it: an Isolation Forest on the session vector, and a CUSUM change-detector on the residual for a second opinion and a plain-English explanation."

8 Demo intro

One point: set expectations, then switch to the dashboard.

- "Four keystrokes, four attacks. Watch the station grid and the Risk Index bar on each card."
- "The last one is the one to watch — no rule will fire, and the system still catches it."

▢▢ – Driver switches to the live dashboard now. Leave the deck; you'll come back for slide 9.

Live-demo runbook

Operator presses keys in `cli/attack`. Always press `r` and wait for the fleet to go green again between attacks.

Attack 1 — `1` — `meter_spoof`

EXPECTED	CP-03 starts reporting ~5 kW while the battery keeps drawing ~120 kW. Within a few seconds the energy residual crosses the fraud threshold → <code>R6_METER_FRAUD</code> → card goes red, frame dropped, Quarantined badge, audio alert. Risk Index on CP-03 also spikes (Tier-2 agrees).
----------	--

- "CP-03 is now claiming five kilowatts. The battery says a hundred and twenty. Those two numbers have to reconcile over time — and they don't."
- "R6 caught it, the proxy dropped the frame, told the charger to go offline, and cut the socket. That station is contained."

IF IT MISBEHAVES — If it doesn't quarantine within ~8 s, say: "the residual is accumulating — there's the trip" as it fires; if it truly stalls, note "R6 fires on sustained under-reporting" and move to `r` then attack 2. Don't wait on camera.

Attack 2 — `r`, then `2` — `fleet_oscillate`

EXPECTED	All stations begin rapid start/stop. The transformer load gauge visibly swings. After ≥ 6 transitions in 10 s → <code>R3_OSCILLATION</code> fires fleet-wide (one badge, not eight).
----------	---

- "This is a coordinated attack — synchronised start/stop across the cluster to swing local load and trip a distribution transformer."
- "R3 counts transitions in a sliding window. It's fleet-scoped on purpose — one finding, not noise."

IF IT MISBEHAVES — If the gauge swing is subtle, point at the transition count / the R3 badge instead of the gauge.

Attack 3 — `r`, then `3` — `session_shadow`

EXPECTED	A second WebSocket opens to a charger ID that already has a live session. The proxy refuses it at the handshake → <code>R4_SESSION_UNIQUE</code> . The legitimate session is untouched.
----------	---

- "Someone just opened a parallel connection claiming to be CP-01, which is already online. That's a session hijack attempt."
- "The proxy keeps a registry of live connections and rejects the duplicate before a single frame is exchanged. The real charger never notices."

Attack 4 — **r** , then **4** — subtle_drift (the pitch)

EXPECTED	One station begins under-reporting power by a compounding ~2% per tick. No Tier-1 rule fires — every value stays legal. The Risk Index bar on that card rises and crosses 0.65 → amber ML ANOMALY badge.
----------	--

- "Every number this station is sending is inside spec. No rule can touch it."
- "But the energy residual is climbing, and the model sees it. Watch the Risk Index — there it goes past the line. That's the attack a threshold engine can't catch, caught."

IF IT MISBEHAVES – Known risk: the drift currently trips fast (~2–3 s in the benchmark), not the slow ~40 s climb. If it jumps quickly, say *"the score climbs and crosses the threshold — no rule involved"* and don't dwell. Do **not** promise a 40-second climb on stage until the drift rate is tuned.

Close the demo — **r**

DO – Press **r** once more so the fleet is green when you cut back to the deck. Then click **Export incident logs** — a JSON file downloads. "Every detection is on the wire and in the forensic log."

Known risks & recovery

RISK	RECOVERY
A socket drops mid-demo	The dashboard auto-reconnects every 2 s — keep talking, it comes back. If it doesn't in ~5 s, cut to the backup video.
Audio alert silent	You forgot the Start Monitoring click in pre-flight. Don't stop to fix it — the visual is enough.
A process died	Operator says so quietly; Narrator keeps on the deck; restart in order CSMS → proxy → twin, or cut to the backup video.
Drift trips too fast	Use the fallback line above. It still proves the point — Tier-2 caught what Tier-1 can't.
"Reset" leaves a station stuck	Press r again; it re-runs Boot → Authorize → StartTransaction and resynchronises both sides.

Q&A preparation

"Isn't your simulation too simple? Is any of this real?"

The **protocol and the detection are real** — real OCPP 1.6-J framing, real call correlation, real session state machine, the actual rule and ML logic, the real quarantine mechanics. What's simulated is the **charger physics** and the **CSMS backend** — and the backend is a drop-in: point the proxy at SteVe or CitrineOS, both real open-source management systems, and nothing changes on our side. The attacks are synthesised because there is no public labelled corpus of OCPP attacks — that's true of the whole field.

"Why an Isolation Forest? Why not a deep model / LSTM?"

The reasoning architecture is the **two tiers**, not the estimator. For a five-dimensional conservation-law residual, a small model we can prove correct beats a big one we can't — and an LSTM in the forwarding path costs 100–1000× the inference latency, which breaks the "inline, sub-millisecond" claim. If we ever add a temporal model it runs out-of-band on a mirrored stream, off the critical path.

"Does this actually scale?"

Tier-1 rule evaluation is ~9 microseconds per frame, measured over 200k frames. One instance sustains thousands of frames per second; a 10,000-charger operator at one meter reading every 10 seconds is about 1,000 frames/sec — a handful of instances. Horizontal scale is charger-ID consistent hashing across replicas with the registries in Redis.

"What about TLS and authentication?"

Out of scope for a 48-hour localhost build, deliberately. In production VoltSentry terminates TLS and enforces the OCPP 2.0.1 security profiles — that's on the roadmap slide. The detection value is orthogonal to transport security: an authenticated charger can still be compromised and lie.

"How is this different from a web application firewall?"

A WAF doesn't speak OCPP. VoltSentry enforces the **session state machine** (you can't send meter values before you're authorised), **physical invariants** (power can't exceed the charger's rating), and **energy conservation** across a session. Those are domain rules a generic firewall has no concept of.

"How does R6 catch meter fraud without false-positiving honest sessions?"

It requires *both* a meaningful accumulated residual *and* a residual that is still climbing fast right now. A brief measurement wobble produces neither. Sustained under-reporting produces both, and the gap only widens.

"What's your false-positive rate?"

Be honest: the current benchmark is on a small recorded fixture, so the FPR figure isn't meaningful yet. The design target is zero Tier-1 false positives (they're deterministic invariants) and Tier-2 tuned against a real dataset — ACN-Data or ElaadNL — before any production claim. The harness to measure it exists; the dataset is the next step.

"Who buys this, and how is it deployed?"

Charge-point operators (CPOs) and CSMS vendors. It deploys as an ingress sidecar in front of the existing CSMS — no charger firmware change, no backend change. That's the entire commercial pitch: security you can add without touching the fleet.

"What's genuinely novel here?"

Inline, transport-layer, OCPP-aware detection with active quarantine — and specifically the energy-conservation residual as a feature that makes slow billing fraud detectable from the wire alone, without ground-truth power measurement.

VoltSentry · CodeToCreate, VIT Vellore · Generated for the team. Numbers in this guide trace to the codebase and its test suite; update them if the code changes before you present.